COMPREHENSIVE JAVA SWING TUTORIAL

From GUI Fundamentals to a Complete Information System

Language	Java
GUI Toolkit	Swing
Recommended IDE	Apache NetBeans
Database	MySQL 8.x for JDBC stages
Examples	90 complete runnable programs
Target	Beginner to advanced desktop application development

Roadmap

Stage	Topic	Primary Outcome
1	JFrame / JLabel	Create windows, labels, layouts, fonts, colors and basic GUI structure.
2	JTextField / JButton	Collect input and respond to button events.
3	Conditions + validation	Validate data and give user-friendly error feedback.
4	JTextArea / JOptionPane	Work with multiline text and dialogs.
5	JCheckBox / JRadioButton	Collect multiple-choice and single-choice options.
6	JComboBox / JList	Present selectable data collections.
7	JTable	Display structured tabular data.
8	CRUD using JTable	Create, read, update and delete in-memory records.
9	Menus / multiple forms	Build navigation and multi-window applications.
10	File handling	Persist and reload application data using files.
11	OOP Models	Separate domain data from the GUI.
12	ArrayList	Manage dynamic object collections.
13	JDBC	Connect Swing applications to MySQL.
14	Database CRUD	Perform persistent CRUD operations.
15	Search/filtering	Find and filter data efficiently.
16	MVC/DAO	Apply maintainable application architecture.
17	SwingWorker	Run slow tasks without freezing the GUI.
18	Complete Information System	Combine GUI, OOP, DAO, JDBC, validation and reporting concepts.

Stage 1: JFrame / JLabel

JFrame is the top-level Swing window. JLabel displays non-editable text or icons. Teach close operation, sizing, centering, layout managers, fonts, colors and borders.

Discussion

Key concepts: JFrame lifecycle; setDefaultCloseOperation; setSize vs pack; setLocationRelativeTo; JLabel alignment; layout managers; Event Dispatch Thread.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Hello Swing

Goal: Create the first JFrame and JLabel.

```
import javax.swing.*;
import java.awt.*;

public class W01E01HelloSwing {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Hello Swing");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JLabel label = new JLabel("Welcome to Java Swing!", SwingConstants.CENTER);
            label.setFont(new Font("SansSerif", Font.BOLD, 24));
            frame.add(label);
            frame.setVisible(true);
        });
    }
}
```

Example 2: Student Profile Labels

Goal: Arrange several labels with GridLayout.

```
import javax.swing.*;
import java.awt.*;

public class W01E02Profile {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Student Profile");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            frame.setLayout(new GridLayout(4, 1, 5, 5));
            frame.add(new JLabel("STUDENT PROFILE", SwingConstants.CENTER));
            frame.add(new JLabel("Name: Alex Santos", SwingConstants.CENTER));
            frame.add(new JLabel("Course: BSIT", SwingConstants.CENTER));
            frame.add(new JLabel("Year Level: 2", SwingConstants.CENTER));
            frame.setVisible(true);
        });
    }
```

```
}
```

Example 3: Styled Status Labels

Goal: Apply colors, borders and fonts.

```
import javax.swing.*;
import java.awt.*;

public class W01E03Status {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("System Status");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel panel = new JPanel(new GridLayout(1, 3, 10, 10));
            String[] text = {"READY", "PROCESSING", "COMPLETE"};
            Color[] colors = {new Color(220,240,255), new Color(255,245,200), new Color(220,255,220)};
            for (int i=0;i<text.length;i++) {
                JLabel l = new JLabel(text[i], SwingConstants.CENTER);
                l.setOpaque(true); l.setBackground(colors[i]);
                l.setFont(new Font("SansSerif", Font.BOLD, 16));
                l.setBorder(BorderFactory.createLineBorder(Color.GRAY));
                panel.add(l);
            }
            frame.add(panel);
            frame.setVisible(true);
        });
    }
}
```

Example 4: BorderLayout Dashboard

Goal: Place labels in different regions.

```
import javax.swing.*;
import java.awt.*;

public class W01E04Layout {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("BorderLayout Demo");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            frame.setLayout(new BorderLayout(8,8));
            frame.add(new JLabel("HEADER", SwingConstants.CENTER), BorderLayout.NORTH);
            frame.add(new JLabel("Navigation", SwingConstants.CENTER), BorderLayout.WEST);
            frame.add(new JLabel("Main Content Area", SwingConstants.CENTER), BorderLayout.CENTER);
            frame.add(new JLabel("Status: Ready", SwingConstants.CENTER), BorderLayout.SOUTH);
            frame.setVisible(true);
        });
    }
}
```

Example 5: HTML Label

Goal: Use simple HTML formatting in JLabel.

```
import javax.swing.*;
import java.awt.*;

public class W01E05HtmlLabel {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("HTML JLabel");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
```

```
            JLabel label = new JLabel("<html><div style='text-align:center'><h2>Java Swing</h2><p>Desktop GUI
Development</p><b>Stage 1 Complete</b></div></html>", SwingConstants.CENTER);
            frame.add(label);
            frame.setVisible(true);
        });
    }
}
```

Stage Review Questions

6. What problem does JFrame / JLabel solve in a Swing application?
1. Which events or methods are most important in these examples?
2. What invalid user actions should the application prevent?
3. How would you improve the layout and usability of one example?
4. How can this stage be combined with the previous stage in a larger system?

Stage 2: JTextField / JButton

JTextField accepts single-line input and JButton initiates actions. Teach event-driven programming with ActionListener and basic parsing.

Discussion

Key concepts: getText/setText; button actions; lambdas; focus; JPasswordField; parsing strings to numbers.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Greeting Form

Goal: Read text and update a label when a button is clicked

```
import javax.swing.*;
import java.awt.*;

public class W02E01Greeting {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Greeting Form");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p = new JPanel(new FlowLayout());
            JTextField name = new JTextField(18);
            JButton greet = new JButton("Greet");
            JLabel output = new JLabel("Enter your name.");
            greet.addActionListener(e -> output.setText("Hello, " + name.getText() + "!"));
            p.add(new JLabel("Name:")); p.add(name); p.add(greet); p.add(output);
            frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Example 2: Add Two Numbers

Goal: Convert JTextField input to numeric data.

```
import javax.swing.*;
import java.awt.*;

public class W02E02Adder {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Number Adder");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p = new JPanel(new GridLayout(4,2,6,6));
            JTextField a = new JTextField(); JTextField b = new JTextField();
            JButton add = new JButton("Add"); JLabel result = new JLabel("0");
            add.addActionListener(e -> {
                double x = Double.parseDouble(a.getText());
```

```
                double y = Double.parseDouble(b.getText());
                result.setText(String.valueOf(x + y));
            });
            p.add(new JLabel("First:")); p.add(a); p.add(new JLabel("Second:")); p.add(b);
            p.add(add); p.add(new JLabel()); p.add(new JLabel("Result:")); p.add(result);
            frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Example 3: Temperature Converter

Goal: Perform a calculation from user input.

```
import javax.swing.*;
import java.awt.*;

public class W02E03Temperature {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Celsius to Fahrenheit");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p = new JPanel(new FlowLayout());
            JTextField c = new JTextField(10); JButton convert = new JButton("Convert"); JLabel out = new JLabel();
            convert.addActionListener(e -> { double v=Double.parseDouble(c.getText());
out.setText(String.format("%.2f F", v*9/5+32)); });
            p.add(new JLabel("Celsius:")); p.add(c); p.add(convert); p.add(out); frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Example 4: Clear Form

Goal: Use multiple buttons and clear fields.

```
import javax.swing.*;
import java.awt.*;

public class W02E04ClearForm {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Simple Form");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p = new JPanel(new GridLayout(4,2,6,6));
            JTextField first = new JTextField(); JTextField last = new JTextField();
            JButton show = new JButton("Show Full Name"); JButton clear = new JButton("Clear"); JLabel out = new
JLabel();
            show.addActionListener(e -> out.setText(first.getText()+" "+last.getText()));
            clear.addActionListener(e -> { first.setText(""); last.setText(""); out.setText("");
first.requestFocus(); });
            p.add(new JLabel("First name:"));p.add(first);p.add(new JLabel("Last
name:"));p.add(last);p.add(show);p.add(clear);p.add(new JLabel("Output:"));p.add(out);frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Example 5: Mini Login

Goal: Handle password-like input and events.

```
import javax.swing.*;
import java.awt.*;

public class W02E05Login {
```

```
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Mini Login");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p = new JPanel(new GridLayout(4,2,6,6));
            JTextField user = new JTextField(); JPasswordField pass = new JPasswordField(); JButton login = new
JButton("Login"); JLabel status = new JLabel();
            login.addActionListener(e -> status.setText("Submitted for user: " + user.getText()));
            p.add(new JLabel("Username:"));p.add(user);p.add(new
JLabel("Password:"));p.add(pass);p.add(login);p.add(new JLabel());p.add(new
JLabel("Status:"));p.add(status);frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Stage Review Questions

1. What problem does JTextField / JButton solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 3: Conditions + Validation

Validation prevents incorrect or incomplete data from entering the application. Combine conditions, parsing and exception handling.

Discussion

Key concepts: trim/isEmpty; numeric ranges; if/else-if; regex; try/catch; validation order; actionable error messages.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Required Field Validation

Goal: Reject blank input before processing.

```
import javax.swing.*;
import java.awt.*;

public class W03E01Required {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Required Field");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p=new JPanel(new FlowLayout()); JTextField name=new JTextField(15); JButton save=new
JButton("Save"); JLabel msg=new JLabel();
            save.addActionListener(e->{ String v=name.getText().trim(); if(v.isEmpty()) msg.setText("Name is
required."); else msg.setText("Saved: "+v); });
            p.add(new JLabel("Name:"));p.add(name);p.add(save);p.add(msg);frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Example 2: Age Range Validation

Goal: Validate integers and acceptable ranges.

```
import javax.swing.*;
import java.awt.*;

public class W03E02Age {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Age Validator");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p=new JPanel(new FlowLayout()); JTextField age=new JTextField(8); JButton check=new
JButton("Check"); JLabel msg=new JLabel();
            check.addActionListener(e->{ try { int a=Integer.parseInt(age.getText()); msg.setText(a>=18 && a<=120 ?
"Valid adult age" : "Age must be 18-120"); } catch(NumberFormatException x){ msg.setText("Enter a whole number.");
}});
            p.add(new JLabel("Age:"));p.add(age);p.add(check);p.add(msg);frame.add(p);
```

```
            frame.setVisible(true);
        });
    }
}
```

Example 3: Grade Classification

Goal: Use if/else-if conditions after validation.

```
import javax.swing.*;
import java.awt.*;

public class W03E03Grade {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Grade Classifier");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p=new JPanel(new FlowLayout()); JTextField grade=new JTextField(8); JButton check=new
JButton("Evaluate"); JLabel msg=new JLabel();
            check.addActionListener(e->{ try { double g=Double.parseDouble(grade.getText()); if(g<0||g>100)
msg.setText("0-100 only"); else if(g>=90)msg.setText("Excellent"); else if(g>=75)msg.setText("Passed"); else
msg.setText("Failed"); }catch(NumberFormatException x){msg.setText("Numeric grade required");}});
            p.add(new JLabel("Grade:"));p.add(grade);p.add(check);p.add(msg);frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Example 4: Email Validation

Goal: Apply simple pattern validation.

```
import javax.swing.*;
import java.awt.*;

public class W03E04Email {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Email Validator");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p=new JPanel(new FlowLayout()); JTextField email=new JTextField(22); JButton check=new
JButton("Validate"); JLabel msg=new JLabel();
            check.addActionListener(e->{ String v=email.getText().trim(); boolean ok=v.matches("^[A-Za-z0-9+_.-
]+@[A-Za-z0-9.-]+$"); msg.setText(ok?"Email format accepted":"Invalid email format");});
            p.add(new JLabel("Email:"));p.add(email);p.add(check);p.add(msg);frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Example 5: Registration Validation

Goal: Combine several validation rules.

```
import javax.swing.*;
import java.awt.*;

public class W03E05Registration {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Registration Validation");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p=new JPanel(new GridLayout(5,2,6,6)); JTextField user=new JTextField();JPasswordField pass=new
JPasswordField();JTextField age=new JTextField();JButton register=new JButton("Register");JLabel msg=new JLabel();
```

```
            register.addActionListener(e->{ String u=user.getText().trim();String pw=new String(pass.getPassword());
try{int a=Integer.parseInt(age.getText()); if(u.length()<4)msg.setText("Username min 4 chars"); else
if(pw.length()<8)msg.setText("Password min 8 chars"); else if(a<13)msg.setText("Must be at least 13"); else
msg.setText("Registration valid");}catch(NumberFormatException x){msg.setText("Age must be numeric");}});
            p.add(new JLabel("Username:"));p.add(user);p.add(new JLabel("Password:"));p.add(pass);p.add(new
JLabel("Age:"));p.add(age);p.add(register);p.add(new JLabel());p.add(new JLabel("Status:"));p.add(msg);frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Stage Review Questions

1. What problem does Conditions + Validation solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 4: JTextArea / JOptionPane

JTextArea handles multiline text. JOptionPane provides ready-made dialogs for messages, input and confirmation.

Discussion

Key concepts: JScrollPane; line wrapping; append; showMessageDialog; showInputDialog; showConfirmDialog.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Notes Editor

Goal: Use JTextArea inside JScrollPane.

```
import javax.swing.*;
import java.awt.*;

public class W04E01Notes {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Notes Editor");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JTextArea area=new JTextArea(10,35); area.setLineWrap(true);area.setWrapStyleWord(true); frame.add(new
JScrollPane(area));
            frame.setVisible(true);
        });
    }
}
```

Example 2: Word Counter

Goal: Analyze multiline input.

```
import javax.swing.*;
import java.awt.*;

public class W04E02WordCount {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Word Counter");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p=new JPanel(new BorderLayout()); JTextArea area=new JTextArea(); JButton count=new
JButton("Count Words"); JLabel out=new JLabel("Words: 0");
            count.addActionListener(e->{String s=area.getText().trim(); int n=s.isEmpty()?0:s.split("\s+").length;
out.setText("Words: "+n);}); p.add(new
JScrollPane(area),BorderLayout.CENTER);p.add(count,BorderLayout.SOUTH);p.add(out,BorderLayout.NORTH);frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Example 3: Information Dialog

Goal: Display messages with JOptionPane.

```
import javax.swing.*;
import java.awt.*;

public class W04E03InfoDialog {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Information Dialog");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JButton b=new JButton("Show Message"); b.addActionListener(e->JOptionPane.showMessageDialog(frame,"Swing
dialogs are easy to use.","Information",JOptionPane.INFORMATION_MESSAGE)); frame.add(b);
            frame.setVisible(true);
        });
    }
}
```

Example 4: Confirmation Dialog

Goal: Ask for confirmation before an action.

```
import javax.swing.*;
import java.awt.*;

public class W04E04Confirm {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Confirmation");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JButton b=new JButton("Delete Record"); b.addActionListener(e->{int
r=JOptionPane.showConfirmDialog(frame,"Delete this record?","Confirm",JOptionPane.YES_NO_OPTION);
if(r==JOptionPane.YES_OPTION)JOptionPane.showMessageDialog(frame,"Record deleted.");});frame.add(b);
            frame.setVisible(true);
        });
    }
}
```

Example 5: Text Analyzer

Goal: Combine JTextArea and input dialogs.

```
import javax.swing.*;
import java.awt.*;

public class W04E05Analyzer {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Text Analyzer");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JTextArea area=new JTextArea(); JButton add=new JButton("Add Line"); JPanel p=new JPanel(new
BorderLayout());
            add.addActionListener(e->{String s=JOptionPane.showInputDialog(frame,"Enter a line:");
if(s!=null&&!s.trim().isEmpty()) area.append(s+System.lineSeparator());}); p.add(new
JScrollPane(area),BorderLayout.CENTER);p.add(add,BorderLayout.SOUTH);frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Stage Review Questions

1. What problem does JTextArea / JOptionPane solve in a Swing application?

2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 5: JCheckBox / JRadioButton

JCheckBox supports independent choices. JRadioButton normally belongs to ButtonGroup for mutually exclusive choices.

Discussion

Key concepts: isSelected; ButtonGroup; default selections; combining controls; calculating based on options.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Hobby Selector

Goal: Read several JCheckBox values.

```
import javax.swing.*;
import java.awt.*;

public class W05E01CheckBox {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Hobbies");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p=new JPanel(new GridLayout(5,1)); JCheckBox a=new JCheckBox("Programming"),b=new
JCheckBox("Gaming"),c=new JCheckBox("Reading"); JButton show=new JButton("Show"); JLabel out=new JLabel();
            show.addActionListener(e->{String s="Selected:"; if(a.isSelected())s+="
Programming";if(b.isSelected())s+=" Gaming";if(c.isSelected())s+="
Reading";out.setText(s);});p.add(a);p.add(b);p.add(c);p.add(show);p.add(out);frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Example 2: Gender Radio Buttons

Goal: Group mutually exclusive radio buttons.

```
import javax.swing.*;
import java.awt.*;

public class W05E02Radio {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Radio Buttons");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p=new JPanel(new GridLayout(4,1)); JRadioButton a=new JRadioButton("Option A"),b=new
JRadioButton("Option B");ButtonGroup g=new ButtonGroup();g.add(a);g.add(b);JButton show=new JButton("Show");JLabel
out=new JLabel();
            show.addActionListener(e->out.setText(a.isSelected()?"Option A":b.isSelected()?"Option B":"Nothing
selected"));p.add(a);p.add(b);p.add(show);p.add(out);frame.add(p);
            frame.setVisible(true);
        });
```

```
    }
}
```

Example 3: Pizza Order

Goal: Calculate options selected by checkboxes.

```
import javax.swing.*;
import java.awt.*;

public class W05E03Pizza {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Pizza Order");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p=new JPanel(new GridLayout(6,1));JCheckBox cheese=new JCheckBox("Extra cheese +50"),pep=new
JCheckBox("Pepperoni +70"),mush=new JCheckBox("Mushroom +40");JButton total=new JButton("Compute Total");JLabel
out=new JLabel();
            total.addActionListener(e->{int
x=200;if(cheese.isSelected())x+=50;if(pep.isSelected())x+=70;if(mush.isSelected())x+=40;out.setText("Total:
P"+x);});p.add(new JLabel("Base pizza:
P200"));p.add(cheese);p.add(pep);p.add(mush);p.add(total);p.add(out);frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Example 4: Payment Method

Goal: Use radio buttons to choose one payment method.

```
import javax.swing.*;
import java.awt.*;

public class W05E04Payment {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Payment Method");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p=new JPanel(new GridLayout(5,1));JRadioButton cash=new JRadioButton("Cash"),card=new
JRadioButton("Card"),gcash=new JRadioButton("GCash");ButtonGroup g=new
ButtonGroup();g.add(cash);g.add(card);g.add(gcash);JButton pay=new JButton("Continue");JLabel out=new
JLabel();pay.addActionListener(e->{String
s=cash.isSelected()?"Cash":card.isSelected()?"Card":gcash.isSelected()?"GCash":"None";out.setText("Payment:
"+s);});p.add(cash);p.add(card);p.add(gcash);p.add(pay);p.add(out);frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Example 5: Survey Form

Goal: Combine checkboxes and radio buttons.

```
import javax.swing.*;
import java.awt.*;

public class W05E05Survey {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Survey");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p=new JPanel(new GridLayout(7,1));JRadioButton beginner=new JRadioButton("Beginner"),advanced=new
JRadioButton("Advanced");ButtonGroup g=new ButtonGroup();g.add(beginner);g.add(advanced);JCheckBox java=new
JCheckBox("Java"),python=new JCheckBox("Python");JButton submit=new JButton("Submit");JLabel out=new
```

```
JLabel();submit.addActionListener(e->{String
level=beginner.isSelected()?"Beginner":advanced.isSelected()?"Advanced":"Not selected";out.setText("Level="+level+",
Java="+java.isSelected()+",
Python="+python.isSelected());});p.add(beginner);p.add(advanced);p.add(java);p.add(python);p.add(submit);p.add(out);
frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Stage Review Questions

1. What problem does JCheckBox / JRadioButton solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 6: JComboBox / JList

These controls present collections of choices. JComboBox is compact; JList is better when users need to see several items or make multiple selections.

Discussion

Key concepts: models; getSelectedItem; getSelectedValue(s); selection modes; listeners; dependent lists.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Course Combo Box

Goal: Read a JComboBox selection.

```
import javax.swing.*;
import java.awt.*;

public class W06E01Combo {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Course Selector");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p=new JPanel(new FlowLayout());JComboBox<String> combo=new JComboBox<>(new
String[]{"BSIT","BSCS","BSEMC"});JButton show=new JButton("Show");JLabel out=new JLabel();show.addActionListener(e-
>out.setText("Course: "+combo.getSelectedItem()));p.add(combo);p.add(show);p.add(out);frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Example 2: City List

Goal: Read a JList selection.

```
import javax.swing.*;
import java.awt.*;

public class W06E02List {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("City List");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JList<String> list=new JList<>(new String[]{"Manila","Quezon
City","Makati","Pasig","Taguig"});list.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);JLabel out=new
JLabel("Choose a city");list.addListSelectionListener(e->{if(!e.getValueIsAdjusting())out.setText("Selected:
"+list.getSelectedValue());});frame.add(new
JScrollPane(list),BorderLayout.CENTER);frame.add(out,BorderLayout.SOUTH);
            frame.setVisible(true);
        });
    }
}
```

Example 3: Multiple Skill List

Goal: Allow multiple JList selections.

```
import javax.swing.*;
import java.awt.*;

public class W06E03MultiList {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Skills");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JList<String> list=new JList<>(new
String[]{"Java","Python","C++","SQL","JavaScript"});list.setSelectionMode(ListSelectionModel.MULTIPLE_INTERVAL_SELEC
TION);JButton show=new JButton("Show Skills");JLabel out=new JLabel();show.addActionListener(e->out.setText("Skills:
"+String.join(", ",list.getSelectedValuesList())));frame.add(new
JScrollPane(list),BorderLayout.CENTER);frame.add(show,BorderLayout.SOUTH);frame.add(out,BorderLayout.NORTH);
            frame.setVisible(true);
        });
    }
}
```

Example 4: Dependent Combo Boxes

Goal: Update one combo based on another.

```
import javax.swing.*;
import java.awt.*;

public class W06E04DependentCombo {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Dependent Combos");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p=new JPanel(new GridLayout(3,2,5,5));JComboBox<String> country=new JComboBox<>(new
String[]{"Philippines","Japan"});JComboBox<String> city=new JComboBox<>();Runnable load=()-
>{city.removeAllItems();String c=(String)country.getSelectedItem();String[] v=c.equals("Philippines")?new
String[]{"Manila","Cebu","Davao"}:new String[]{"Tokyo","Osaka","Kyoto"};for(String
s:v)city.addItem(s);};country.addActionListener(e->load.run());load.run();p.add(new
JLabel("Country:"));p.add(country);p.add(new JLabel("City:"));p.add(city);frame.add(p);
            frame.setVisible(true);
        });
    }
}
```

Example 5: Enrollment Selection

Goal: Combine JComboBox and JList in one form.

```
import javax.swing.*;
import java.awt.*;

public class W06E05Enrollment {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Enrollment Selector");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JPanel p=new JPanel(new BorderLayout());JComboBox<String> year=new JComboBox<>(new String[]{"1st
Year","2nd Year","3rd Year","4th Year"});JList<String> subjects=new JList<>(new
String[]{"Programming","Database","Networking","OOP"});JButton enroll=new JButton("Enroll");JTextArea out=new
JTextArea();enroll.addActionListener(e->out.setText("Year: "+year.getSelectedItem()+"
Subjects: "+subjects.getSelectedValuesList()));p.add(year,BorderLayout.NORTH);p.add(new
JScrollPane(subjects),BorderLayout.CENTER);p.add(enroll,BorderLayout.SOUTH);frame.add(p,BorderLayout.CENTER);frame.a
dd(new JScrollPane(out),BorderLayout.EAST);
            frame.setVisible(true);
```

```
        });
    }
}
```

Stage Review Questions

1. What problem does JComboBox / JList solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 7: JTable

JTable displays structured rows and columns. Most real applications use DefaultTableModel or a custom TableModel.

Discussion

Key concepts: JTable; DefaultTableModel; JScrollPane; row selection; cell editability; adding rows; reading cell values.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Basic Student Table

Goal: Display rows and columns in JTable.

```
import javax.swing.*;
import java.awt.*;

public class W07E01BasicTable {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Stage 7 - E01BasicTable");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            String[] columns={"ID","Name","Course"};
            String[][] data={
                {"1","Ana","BSIT"},
                {"2","Ben","BSCS"},
                {"3","Cara","BSEMC"}
            };
            JTable table=new JTable(data,columns);

            frame.add(new JScrollPane(table));
            frame.setVisible(true);
        });
    }
}
```

Example 2: Table Selection

Goal: Read the selected table row.

```
import javax.swing.*;
import java.awt.*;

public class W07E02Selection {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Table Selection");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            String[] c={"ID","Product","Price"};Object[][]
d={{1,"Keyboard",850},{2,"Mouse",450},{3,"Monitor",7200}};JTable table=new JTable(d,c);JLabel out=new JLabel("Select
```

```
a row");table.getSelectionModel().addListSelectionListener(e->{int
r=table.getSelectedRow();if(r>=0)out.setText("Selected: "+table.getValueAt(r,1));});frame.add(new
JScrollPane(table),BorderLayout.CENTER);frame.add(out,BorderLayout.SOUTH);
            frame.setVisible(true);
        });
    }
}
```

Example 3: Non-editable Model

Goal: Create a DefaultTableModel that prevents cell editing.

```
import javax.swing.*;
import java.awt.*;

public class W07E03ReadOnly {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Read-only JTable");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            String[] c={"Code","Subject","Units"};Object[][]
d={{"IT101","Programming",3},{"IT102","Database",3}};javax.swing.table.DefaultTableModel model=new
javax.swing.table.DefaultTableModel(d,c){public boolean isCellEditable(int r,int col){return false;}};JTable
table=new JTable(model);frame.add(new JScrollPane(table));
            frame.setVisible(true);
        });
    }
}
```

Example 4: Add Rows to Table

Goal: Add data dynamically to DefaultTableModel.

```
import javax.swing.*;
import java.awt.*;

public class W07E04AddRow {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Add JTable Row");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            javax.swing.table.DefaultTableModel model=new javax.swing.table.DefaultTableModel(new
String[]{"Name","Score"},0);JTable table=new JTable(model);JTextField name=new JTextField(10),score=new
JTextField(6);JButton add=new JButton("Add");JPanel top=new JPanel();add.addActionListener(e->{model.addRow(new
Object[]{name.getText(),score.getText()});name.setText("");score.setText("");});top.add(name);top.add(score);top.add
(add);frame.add(top,BorderLayout.NORTH);frame.add(new JScrollPane(table),BorderLayout.CENTER);
            frame.setVisible(true);
        });
    }
}
```

Example 5: Table Totals

Goal: Compute totals from table data.

```
import javax.swing.*;
import java.awt.*;

public class W07E05Totals {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Table Totals");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            Object[][] d={{"Pen",10,15.0},{"Notebook",5,40.0},{"Bag",2,600.0}};String[]
c={"Item","Qty","Price"};JTable table=new JTable(d,c);JButton total=new JButton("Compute Total");JLabel out=new
```

```
JLabel();total.addActionListener(e->{double sum=0;for(int
r=0;r<table.getRowCount();r++)sum+=(Integer)table.getValueAt(r,1)*(Double)table.getValueAt(r,2);out.setText("Grand
Total: P"+sum);});frame.add(new JScrollPane(table),BorderLayout.CENTER);JPanel south=new
JPanel();south.add(total);south.add(out);frame.add(south,BorderLayout.SOUTH);
            frame.setVisible(true);
        });
    }
}
```

Stage Review Questions

1. What problem does JTable solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 8: CRUD using JTable

CRUD means Create, Read, Update and Delete. At this stage students perform CRUD against a table model before introducing a database.

Discussion

Key concepts: row selection; DefaultTableModel mutations; form-to-table mapping; confirmation; clearing forms.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Create Records

Goal: Insert rows into an in-memory table.

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class W08E01Create extends JFrame {
    private final DefaultTableModel model=new DefaultTableModel(new String[]{"ID","Name","Course"},0);
    private final JTable table=new JTable(model);
    private final JTextField id=new JTextField(6),name=new JTextField(12),course=new JTextField(10);
    public W08E01Create(){
        setTitle("Create
Records");setSize(650,400);setDefaultCloseOperation(EXIT_ON_CLOSE);setLocationRelativeTo(null);
        JPanel form=new JPanel();form.add(new JLabel("ID"));form.add(id);form.add(new
JLabel("Name"));form.add(name);form.add(new JLabel("Course"));form.add(course);
        JButton add=new JButton("Add");add.addActionListener(e->model.addRow(new
Object[]{id.getText(),name.getText(),course.getText()}));form.add(add);
        add(form,BorderLayout.NORTH);add(new JScrollPane(table),BorderLayout.CENTER);setVisible(true);
    }
    public static void main(String[] args){SwingUtilities.invokeLater(W08E01Create::new);}
}
```

Example 2: Read/Select Records

Goal: Load the selected row into form fields.

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class W08E02Read extends JFrame {
    private final DefaultTableModel model=new DefaultTableModel(new String[]{"ID","Name","Course"},0);
    private final JTable table=new JTable(model);
    private final JTextField id=new JTextField(6),name=new JTextField(12),course=new JTextField(10);
    public W08E02Read(){
        setTitle("Read Selected
Record");setSize(650,400);setDefaultCloseOperation(EXIT_ON_CLOSE);setLocationRelativeTo(null);
        JPanel form=new JPanel();form.add(new JLabel("ID"));form.add(id);form.add(new
JLabel("Name"));form.add(name);form.add(new JLabel("Course"));form.add(course);
        JButton load=new JButton("Load Selected");load.addActionListener(e->{int
r=table.getSelectedRow();if(r>=0){id.setText(model.getValueAt(r,0).toString());name.setText(model.getValueAt(r,1).to
```

```
String());course.setText(model.getValueAt(r,2).toString());}});form.add(load);model.addRow(new
Object[]{"1","Ana","BSIT"});model.addRow(new Object[]{"2","Ben","BSCS"});
        add(form,BorderLayout.NORTH);add(new JScrollPane(table),BorderLayout.CENTER);setVisible(true);
    }
    public static void main(String[] args){SwingUtilities.invokeLater(W08E02Read::new);}
}
```

Example 3: Update Records

Goal: Modify the currently selected row.

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class W08E03Update extends JFrame {
    private final DefaultTableModel model=new DefaultTableModel(new String[]{"ID","Name","Course"},0);
    private final JTable table=new JTable(model);
    private final JTextField id=new JTextField(6),name=new JTextField(12),course=new JTextField(10);
    public W08E03Update(){
        setTitle("Update
Record");setSize(650,400);setDefaultCloseOperation(EXIT_ON_CLOSE);setLocationRelativeTo(null);
        JPanel form=new JPanel();form.add(new JLabel("ID"));form.add(id);form.add(new
JLabel("Name"));form.add(name);form.add(new JLabel("Course"));form.add(course);
        JButton update=new JButton("Update Selected");update.addActionListener(e->{int
r=table.getSelectedRow();if(r>=0){model.setValueAt(id.getText(),r,0);model.setValueAt(name.getText(),r,1);model.setV
alueAt(course.getText(),r,2);}});form.add(update);model.addRow(new Object[]{"1","Ana","BSIT"});
        add(form,BorderLayout.NORTH);add(new JScrollPane(table),BorderLayout.CENTER);setVisible(true);
    }
    public static void main(String[] args){SwingUtilities.invokeLater(W08E03Update::new);}
}
```

Example 4: Delete Records

Goal: Remove the selected row with confirmation.

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class W08E04Delete extends JFrame {
    private final DefaultTableModel model=new DefaultTableModel(new String[]{"ID","Name","Course"},0);
    private final JTable table=new JTable(model);
    private final JTextField id=new JTextField(6),name=new JTextField(12),course=new JTextField(10);
    public W08E04Delete(){
        setTitle("Delete
Record");setSize(650,400);setDefaultCloseOperation(EXIT_ON_CLOSE);setLocationRelativeTo(null);
        JPanel form=new JPanel();form.add(new JLabel("ID"));form.add(id);form.add(new
JLabel("Name"));form.add(name);form.add(new JLabel("Course"));form.add(course);
        JButton del=new JButton("Delete Selected");del.addActionListener(e->{int
r=table.getSelectedRow();if(r>=0&&JOptionPane.showConfirmDialog(this,"Delete selected
record?")==JOptionPane.YES_OPTION)model.removeRow(r);});form.add(del);model.addRow(new Object[]{"1","Ana","BSIT"});
        add(form,BorderLayout.NORTH);add(new JScrollPane(table),BorderLayout.CENTER);setVisible(true);
    }
    public static void main(String[] args){SwingUtilities.invokeLater(W08E04Delete::new);}
}
```

Example 5: Complete JTable CRUD

Goal: Combine create, read, update and delete.

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class W08E05Crud extends JFrame {
    private final DefaultTableModel model=new DefaultTableModel(new String[]{"ID","Name","Course"},0);
    private final JTable table=new JTable(model);
    private final JTextField id=new JTextField(6),name=new JTextField(12),course=new JTextField(10);
    public W08E05Crud(){
```

```
        setTitle("Complete JTable
CRUD");setSize(650,400);setDefaultCloseOperation(EXIT_ON_CLOSE);setLocationRelativeTo(null);
        JPanel form=new JPanel();form.add(new JLabel("ID"));form.add(id);form.add(new
JLabel("Name"));form.add(name);form.add(new JLabel("Course"));form.add(course);
        JButton add=new JButton("Add"),update=new JButton("Update"),del=new JButton("Delete"),clear=new
JButton("Clear");
        add.addActionListener(e->model.addRow(new Object[]{id.getText(),name.getText(),course.getText()}));
        table.getSelectionModel().addListSelectionListener(e->{int
r=table.getSelectedRow();if(r>=0){id.setText(model.getValueAt(r,0).toString());name.setText(model.getValueAt(r,1).to
String());course.setText(model.getValueAt(r,2).toString());}});
        update.addActionListener(e->{int
r=table.getSelectedRow();if(r>=0){model.setValueAt(id.getText(),r,0);model.setValueAt(name.getText(),r,1);model.setV
alueAt(course.getText(),r,2);}});
        del.addActionListener(e->{int
r=table.getSelectedRow();if(r>=0)model.removeRow(r);});clear.addActionListener(e-
>{id.setText("");name.setText("");course.setText("");table.clearSelection();});
        form.add(add);form.add(update);form.add(del);form.add(clear);
        add(form,BorderLayout.NORTH);add(new JScrollPane(table),BorderLayout.CENTER);setVisible(true);
    }
    public static void main(String[] args){SwingUtilities.invokeLater(W08E05Crud::new);}
}
```

Stage Review Questions

1. What problem does CRUD using JTable solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 9: Menus / Multiple Forms

Large desktop applications require navigation. Menus, multiple windows, CardLayout and internal frames are common approaches.

Discussion

Key concepts: JMenuBar; JMenuItem; dispose vs exit; child windows; CardLayout; JDesktopPane.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Simple Menu Bar

Goal: Create File and Help menus.

```
import javax.swing.*;
import java.awt.*;

public class W09E01Menu {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Menu Demo");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(520, 320);
            frame.setLocationRelativeTo(null);
            JMenuBar bar=new JMenuBar();JMenu file=new JMenu("File"),help=new JMenu("Help");JMenuItem exit=new
JMenuItem("Exit"),about=new JMenuItem("About");exit.addActionListener(e->System.exit(0));about.addActionListener(e-
>JOptionPane.showMessageDialog(frame,"Java Swing Menu
Demo"));file.add(exit);help.add(about);bar.add(file);bar.add(help);frame.setJMenuBar(bar);frame.add(new JLabel("Use
the menu bar above.",SwingConstants.CENTER));
            frame.setVisible(true);
        });
    }
}
```

Example 2: Open Child Form

Goal: Open a second JFrame from a button.

```
import javax.swing.*;
public class W09E02MultipleForms {
 public static void main(String[] args){SwingUtilities.invokeLater(()->{JFrame main=new JFrame("Main
Form");main.setSize(400,200);main.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);JButton open=new JButton("Open
Student Form");open.addActionListener(e->{JFrame child=new JFrame("Student
Form");child.setSize(320,180);child.add(new JLabel("This is another
form.",SwingConstants.CENTER));child.setLocationRelativeTo(main);child.setVisible(true);});main.add(open);main.setLo
cationRelativeTo(null);main.setVisible(true);});}
}
```

Example 3: Login to Dashboard

Goal: Hide login form and show dashboard.

```
import javax.swing.*;import java.awt.*;
public class W09E03LoginDashboard {
```

```
 public static void main(String[] args){SwingUtilities.invokeLater(()->new Login());}
 static class Login extends
JFrame{Login(){setTitle("Login");setSize(350,200);setDefaultCloseOperation(EXIT_ON_CLOSE);JTextField u=new
JTextField();JPasswordField p=new JPasswordField();JButton b=new JButton("Login");setLayout(new
GridLayout(3,2));add(new JLabel("User"));add(u);add(new JLabel("Pass"));add(p);add(b);b.addActionListener(e-
>{if(u.getText().equals("admin")&&new String(p.getPassword()).equals("1234")){dispose();new Dashboard();}else
JOptionPane.showMessageDialog(this,"Invalid login");});setLocationRelativeTo(null);setVisible(true);}}
 static class Dashboard extends
JFrame{Dashboard(){setTitle("Dashboard");setSize(500,300);setDefaultCloseOperation(EXIT_ON_CLOSE);add(new
JLabel("Welcome to Dashboard",SwingConstants.CENTER));setLocationRelativeTo(null);setVisible(true);}}
}
```

Example 4: CardLayout Navigation

Goal: Switch between views in one JFrame.

```
import javax.swing.*;import java.awt.*;
public class W09E04CardLayout extends JFrame{CardLayout cards=new CardLayout();JPanel center=new
JPanel(cards);public W09E04CardLayout(){setTitle("CardLayout
Navigation");setSize(500,300);setDefaultCloseOperation(EXIT_ON_CLOSE);center.add(new
JLabel("HOME",SwingConstants.CENTER),"home");center.add(new
JLabel("STUDENTS",SwingConstants.CENTER),"students");JButton h=new JButton("Home"),s=new
JButton("Students");h.addActionListener(e->cards.show(center,"home"));s.addActionListener(e-
>cards.show(center,"students"));JPanel nav=new
JPanel();nav.add(h);nav.add(s);add(nav,BorderLayout.NORTH);add(center);setLocationRelativeTo(null);setVisible(true);
}public static void main(String[] a){SwingUtilities.invokeLater(W09E04CardLayout::new);}}
```

Example 5: Desktop Pane MDI

Goal: Use JDesktopPane and JInternalFrame.

```
import javax.swing.*;
public class W09E05DesktopPane extends JFrame{JDesktopPane desktop=new JDesktopPane();public
W09E05DesktopPane(){setTitle("MDI Demo");setSize(700,450);setDefaultCloseOperation(EXIT_ON_CLOSE);JMenuBar b=new
JMenuBar();JMenu m=new JMenu("Forms");JMenuItem open=new JMenuItem("Open Student Form");open.addActionListener(e-
>{JInternalFrame f=new JInternalFrame("Student Form",true,true,true,true);f.setSize(300,200);f.add(new
JLabel("Internal
form",SwingConstants.CENTER));f.setVisible(true);desktop.add(f);});m.add(open);b.add(m);setJMenuBar(b);add(desktop);
setLocationRelativeTo(null);setVisible(true);}public static void main(String[]
a){SwingUtilities.invokeLater(W09E05DesktopPane::new);}}
```

Stage Review Questions

1. What problem does Menus / Multiple Forms solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 10: File Handling

File handling gives applications persistence without a database. Teach text files, CSV and file choosers before JDBC.

Discussion

Key concepts: java.nio.file.Files; Path; exceptions; JFileChooser; CSV serialization; loading data into JTable.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Write Text File

Goal: Save JTextArea content using Files.writeString.

```
import javax.swing.*;import java.awt.*;import java.nio.file.*;
public class W10E01SaveText extends JFrame{JTextArea area=new JTextArea();public W10E01SaveText(){setTitle("Save
Text");setSize(500,350);setDefaultCloseOperation(EXIT_ON_CLOSE);JButton save=new JButton("Save
notes.txt");save.addActionListener(e-
>{try{Files.writeString(Path.of("notes.txt"),area.getText());JOptionPane.showMessageDialog(this,"Saved");}catch(Exce
ption x){JOptionPane.showMessageDialog(this,x.getMessage());}});add(new
JScrollPane(area));add(save,BorderLayout.SOUTH);setLocationRelativeTo(null);setVisible(true);}public static void
main(String[] a){SwingUtilities.invokeLater(W10E01SaveText::new);}}
```

Example 2: Read Text File

Goal: Load a text file into JTextArea.

```
import javax.swing.*;import java.awt.*;import java.nio.file.*;
public class W10E02ReadText extends JFrame{JTextArea area=new JTextArea();public W10E02ReadText(){setTitle("Read
Text");setSize(500,350);setDefaultCloseOperation(EXIT_ON_CLOSE);JButton load=new JButton("Load
notes.txt");load.addActionListener(e->{try{area.setText(Files.readString(Path.of("notes.txt")));}catch(Exception
x){JOptionPane.showMessageDialog(this,x.getMessage());}});add(new
JScrollPane(area));add(load,BorderLayout.SOUTH);setLocationRelativeTo(null);setVisible(true);}public static void
main(String[] a){SwingUtilities.invokeLater(W10E02ReadText::new);}}
```

Example 3: File Chooser Save

Goal: Let the user choose where to save.

```
import javax.swing.*;import java.awt.*;import java.nio.file.*;
public class W10E03ChooserSave extends JFrame{JTextArea area=new JTextArea();public
W10E03ChooserSave(){setTitle("Save As");setSize(500,350);setDefaultCloseOperation(EXIT_ON_CLOSE);JButton save=new
JButton("Save As...");save.addActionListener(e->{JFileChooser fc=new
JFileChooser();if(fc.showSaveDialog(this)==JFileChooser.APPROVE_OPTION)try{Files.writeString(fc.getSelectedFile().to
Path(),area.getText());}catch(Exception x){JOptionPane.showMessageDialog(this,x.getMessage());}});add(new
JScrollPane(area));add(save,BorderLayout.SOUTH);setLocationRelativeTo(null);setVisible(true);}public static void
main(String[] a){SwingUtilities.invokeLater(W10E03ChooserSave::new);}}
```

Example 4: CSV Student Records

Goal: Append simple CSV records to a file.

```
import javax.swing.*;import java.awt.*;import java.nio.file.*;import java.nio.charset.StandardCharsets;import
java.nio.file.StandardOpenOption;
public class W10E04CSV extends JFrame{public W10E04CSV(){setTitle("CSV
Writer");setSize(450,220);setDefaultCloseOperation(EXIT_ON_CLOSE);JTextField id=new JTextField(),name=new
JTextField();JButton save=new JButton("Append CSV");setLayout(new GridLayout(3,2));add(new
JLabel("ID"));add(id);add(new JLabel("Name"));add(name);add(save);save.addActionListener(e->{String
line=id.getText()+","+name.getText()+System.lineSeparator();try{Files.writeString(Path.of("students.csv"),line,Stand
ardCharsets.UTF_8,StandardOpenOption.CREATE,StandardOpenOption.APPEND);JOptionPane.showMessageDialog(this,"Saved");}
catch(Exception
x){JOptionPane.showMessageDialog(this,x.getMessage());}});setLocationRelativeTo(null);setVisible(true);}public
static void main(String[] a){SwingUtilities.invokeLater(W10E04CSV::new);}}
```

Example 5: File-based Student Table

Goal: Load CSV rows into JTable.

```
import javax.swing.*;import javax.swing.table.DefaultTableModel;import java.awt.*;import java.nio.file.*;
public class W10E05CSVTable extends JFrame{DefaultTableModel model=new DefaultTableModel(new
String[]{"ID","Name"},0);public W10E05CSVTable(){setTitle("CSV
Table");setSize(500,350);setDefaultCloseOperation(EXIT_ON_CLOSE);JTable table=new JTable(model);JButton load=new
JButton("Load students.csv");load.addActionListener(e->{model.setRowCount(0);try{for(String
line:Files.readAllLines(Path.of("students.csv"))){String[]
p=line.split(",",2);if(p.length==2)model.addRow(p);}}catch(Exception
x){JOptionPane.showMessageDialog(this,x.getMessage());}});add(new
JScrollPane(table));add(load,BorderLayout.SOUTH);setLocationRelativeTo(null);setVisible(true);}public static void
main(String[] a){SwingUtilities.invokeLater(W10E05CSVTable::new);}}
```

Stage Review Questions

1. What problem does File Handling solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 11: OOP Models

A model class represents real application data independently of the user interface. This is the bridge from small demos to maintainable systems.

Discussion

Key concepts: fields; constructors; getters/setters; encapsulation; toString; composition; inheritance.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Student Model

Goal: Create and display a Student object.

```
import javax.swing.*;
class Student { private int id; private String name; private String course; public Student(int id,String name,String
course){this.id=id;this.name=name;this.course=course;} public int getId(){return id;} public String getName(){return
name;} public String getCourse(){return course;} public void setName(String name){this.name=name;} public String
toString(){return id+" - "+name+" ("+course+")";} }
public class W11E01StudentModel{public static void main(String[] a){SwingUtilities.invokeLater(()->{Student s=new
Student(1,"Ana","BSIT");JOptionPane.showMessageDialog(null,s.toString());});}}
```

Example 2: Product Model

Goal: Encapsulate product data and compute inventory value.

```
import javax.swing.*;class Product{private String name;private double price;private int qty;Product(String n,double
p,int q){name=n;price=p;qty=q;}double total(){return price*qty;}public String toString(){return name+"
total=P"+total();}}public class W11E02ProductModel{public static void main(String[]
a){SwingUtilities.invokeLater(()->JOptionPane.showMessageDialog(null,new Product("Keyboard",850,3)));}}
```

Example 3: Model to Form

Goal: Populate Swing labels from an object.

```
import javax.swing.*;import java.awt.*;
class Student { private int id; private String name; private String course; public Student(int id,String name,String
course){this.id=id;this.name=name;this.course=course;} public int getId(){return id;} public String getName(){return
name;} public String getCourse(){return course;} public void setName(String name){this.name=name;} public String
toString(){return id+" - "+name+" ("+course+")";} }
public class W11E03ModelToForm extends JFrame{public W11E03ModelToForm(){Student s=new
Student(7,"Bea","BSCS");setTitle("Student
Model");setSize(350,220);setDefaultCloseOperation(EXIT_ON_CLOSE);setLayout(new GridLayout(3,1));add(new JLabel("ID:
"+s.getId()));add(new JLabel("Name: "+s.getName()));add(new JLabel("Course:
"+s.getCourse()));setLocationRelativeTo(null);setVisible(true);}public static void main(String[]
a){SwingUtilities.invokeLater(W11E03ModelToForm::new);}}
```

Example 4: Form to Model

Goal: Construct an object from form values.

```
import javax.swing.*;import java.awt.*;
```

```
class Student { private int id; private String name; private String course; public Student(int id,String name,String course){this.id=id;this.name=name;this.course=course;} public int getId(){return id;} public String getName(){return name;} public String getCourse(){return course;} public void setName(String name){this.name=name;} public String toString(){return id+" - "+name+" ("+course+")";} }
public class W11E04FormToModel extends JFrame{public W11E04FormToModel(){setTitle("Create Student");setSize(450,250);setDefaultCloseOperation(EXIT_ON_CLOSE);JTextField id=new JTextField(),name=new JTextField(),course=new JTextField();JButton create=new JButton("Create Object");setLayout(new GridLayout(4,2));add(new JLabel("ID"));add(id);add(new JLabel("Name"));add(name);add(new JLabel("Course"));add(course);add(create);create.addActionListener(e->{Student s=new Student(Integer.parseInt(id.getText()),name.getText(),course.getText());JOptionPane.showMessageDialog(this,s);});setLocationRelativeTo(null);setVisible(true);}public static void main(String[] a){SwingUtilities.invokeLater(W11E04FormToModel::new);}}
```

Example 5: Inheritance Models

Goal: Use a base Person class and Student subclass.

```
import javax.swing.*;class Person{protected String name;Person(String n){name=n;}}class CollegeStudent extends Person{private String course;CollegeStudent(String n,String c){super(n);course=c;}public String describe(){return name+" studies "+course;}}public class W11E05Inheritance{public static void main(String[] a){SwingUtilities.invokeLater(()->JOptionPane.showMessageDialog(null,new CollegeStudent("Carlo","BSIT").describe()));}}
```

Stage Review Questions

1. What problem does OOP Models solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 12: ArrayList

ArrayList is a dynamic collection and is useful for managing model objects before introducing persistent storage.

Discussion

Key concepts: add/get/remove; iteration; object lists; synchronization with Swing models; simple searching.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Names ArrayList

Goal: Add and display dynamic string data.

```
import javax.swing.*;import java.awt.*;import java.util.ArrayList;public class W12E01Names extends
JFrame{ArrayList<String> names=new ArrayList<>();DefaultListModel<String> model=new DefaultListModel<>();public
W12E01Names(){setTitle("ArrayList Names");setSize(400,300);setDefaultCloseOperation(EXIT_ON_CLOSE);JTextField
input=new JTextField(15);JButton add=new JButton("Add");JList<String> list=new
JList<>(model);add.addActionListener(e-
>{names.add(input.getText());model.addElement(input.getText());input.setText("");});JPanel top=new
JPanel();top.add(input);top.add(add);add(top,BorderLayout.NORTH);add(new
JScrollPane(list));setLocationRelativeTo(null);setVisible(true);}public static void main(String[]
a){SwingUtilities.invokeLater(W12E01Names::new);}}
```

Example 2: Student Objects in ArrayList

Goal: Store model objects dynamically.

```
import javax.swing.*;import java.util.*;
class Student { private int id; private String name; private String course; public Student(int id,String name,String
course){this.id=id;this.name=name;this.course=course;} public int getId(){return id;} public String getName(){return
name;} public String getCourse(){return course;} public void setName(String name){this.name=name;} public String
toString(){return id+" - "+name+" ("+course+")";} }
public class W12E02StudentList{public static void main(String[] a){SwingUtilities.invokeLater(()-
>{ArrayList<Student> list=new ArrayList<>();list.add(new Student(1,"Ana","BSIT"));list.add(new
Student(2,"Ben","BSCS"));JOptionPane.showMessageDialog(null,list.get(0)+"
"+list.get(1));});}}
```

Example 3: ArrayList to JTable

Goal: Render an object list in a JTable.

```
import javax.swing.*;import javax.swing.table.DefaultTableModel;import java.util.*;import java.awt.*;
class Student { private int id; private String name; private String course; public Student(int id,String name,String
course){this.id=id;this.name=name;this.course=course;} public int getId(){return id;} public String getName(){return
name;} public String getCourse(){return course;} public void setName(String name){this.name=name;} public String
toString(){return id+" - "+name+" ("+course+")";} }
public class W12E03ListTable extends JFrame{public W12E03ListTable(){ArrayList<Student> list=new
ArrayList<>();list.add(new Student(1,"Ana","BSIT"));list.add(new Student(2,"Ben","BSCS"));DefaultTableModel m=new
DefaultTableModel(new String[]{"ID","Name","Course"},0);for(Student s:list)m.addRow(new
Object[]{s.getId(),s.getName(),s.getCourse()});setTitle("ArrayList
JTable");setSize(500,300);setDefaultCloseOperation(EXIT_ON_CLOSE);add(new JScrollPane(new
JTable(m)));setLocationRelativeTo(null);setVisible(true);}public static void main(String[]
a){SwingUtilities.invokeLater(W12E03ListTable::new);}}
```

Example 4: Remove Object

Goal: Delete an object by selected index.

```
import javax.swing.*;import java.awt.*;import java.util.*;public class W12E04Remove extends JFrame{ArrayList<String>
data=new ArrayList<>();DefaultListModel<String> model=new DefaultListModel<>();JList<String> list=new
JList<>(model);public
W12E04Remove(){data.addAll(Arrays.asList("Alpha","Beta","Gamma"));data.forEach(model::addElement);JButton del=new
JButton("Delete Selected");del.addActionListener(e->{int
i=list.getSelectedIndex();if(i>=0){data.remove(i);model.remove(i);}});setTitle("Remove from
ArrayList");setSize(350,300);setDefaultCloseOperation(EXIT_ON_CLOSE);add(new
JScrollPane(list));add(del,BorderLayout.SOUTH);setLocationRelativeTo(null);setVisible(true);}public static void
main(String[] a){SwingUtilities.invokeLater(W12E04Remove::new);}}
```

Example 5: Search ArrayList

Goal: Search objects without a database.

```
import javax.swing.*;import java.awt.*;import java.util.*;
class Student { private int id; private String name; private String course; public Student(int id,String name,String
course){this.id=id;this.name=name;this.course=course;} public int getId(){return id;} public String getName(){return
name;} public String getCourse(){return course;} public void setName(String name){this.name=name;} public String
toString(){return id+" - "+name+" ("+course+")";} }
public class W12E05Search extends JFrame{ArrayList<Student> list=new ArrayList<>();public
W12E05Search(){list.add(new Student(1,"Ana","BSIT"));list.add(new Student(2,"Ben","BSCS"));JTextField q=new
JTextField(15);JButton find=new JButton("Find");JLabel out=new JLabel();find.addActionListener(e->{String
s=q.getText().toLowerCase();String result="Not found";for(Student
st:list)if(st.getName().toLowerCase().contains(s)){result=st.toString();break;}out.setText(result);});JPanel p=new
JPanel();p.add(q);p.add(find);p.add(out);setTitle("ArrayList
Search");setSize(500,200);setDefaultCloseOperation(EXIT_ON_CLOSE);add(p);setLocationRelativeTo(null);setVisible(true
);}public static void main(String[] a){SwingUtilities.invokeLater(W12E05Search::new);}}
```

Stage Review Questions

1. What problem does ArrayList solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 13: JDBC

JDBC is Java's standard database API. Teach connections, SQL execution, ResultSet and PreparedStatement.

Discussion

Key concepts: DriverManager; Connection; Statement; PreparedStatement; ResultSet; try-with-resources; SQL injection prevention.

Database setup used by Stages 13-18:

```
CREATE DATABASE swing_demo;
USE swing_demo;
CREATE TABLE students (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  course VARCHAR(50) NOT NULL,
  email VARCHAR(120)
);
```

Add MySQL Connector/J to the NetBeans project Libraries. Examples assume:
URL: jdbc:mysql://localhost:3306/swing_demo
USER: root
PASSWORD: change_me

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Test JDBC Connection

Goal: Open and close a MySQL connection.

```
import javax.swing.*;import java.sql.*;public class W13E01Connection{static final String
URL="jdbc:mysql://localhost:3306/swing_demo";static final String USER="root";static final String
PASS="change_me";public static void main(String[] a){SwingUtilities.invokeLater(()->{try(Connection
c=DriverManager.getConnection(URL,USER,PASS)){JOptionPane.showMessageDialog(null,"Connected:
"+!c.isClosed());}catch(SQLException e){JOptionPane.showMessageDialog(null,e.getMessage());}});}}
```

Example 2: SELECT Count

Goal: Execute a scalar SELECT query.

```
import javax.swing.*;import java.sql.*;public class W13E02Count{public static void main(String[]
a){SwingUtilities.invokeLater(()->{String url="jdbc:mysql://localhost:3306/swing_demo";try(Connection
c=DriverManager.getConnection(url,"root","change_me");Statement s=c.createStatement();ResultSet
r=s.executeQuery("SELECT COUNT(*) FROM students")){r.next();JOptionPane.showMessageDialog(null,"Students:
"+r.getInt(1));}catch(SQLException e){JOptionPane.showMessageDialog(null,e.getMessage());}});}}
```

Example 3: SELECT to JTextArea

Goal: Display ResultSet data.

```
import javax.swing.*;import java.awt.*;import java.sql.*;public class W13E03SelectText extends JFrame{JTextArea
area=new JTextArea();public W13E03SelectText(){setTitle("JDBC
Select");setSize(500,350);setDefaultCloseOperation(EXIT_ON_CLOSE);JButton load=new JButton("Load
Students");load.addActionListener(e->{area.setText("");try(Connection
c=DriverManager.getConnection("jdbc:mysql://localhost:3306/swing_demo","root","change_me");Statement
s=c.createStatement();ResultSet r=s.executeQuery("SELECT id,name,course FROM
students")){while(r.next())area.append(r.getInt("id")+" | "+r.getString("name")+" | "+r.getString("course")+"
");}catch(SQLException x){area.setText(x.getMessage());}});add(new
JScrollPane(area));add(load,BorderLayout.SOUTH);setLocationRelativeTo(null);setVisible(true);}public static void
main(String[] a){SwingUtilities.invokeLater(W13E03SelectText::new);}}
```

Example 4: Prepared INSERT

Goal: Use PreparedStatement safely.

```
import javax.swing.*;import java.awt.*;import java.sql.*;public class W13E04Insert extends JFrame{public
W13E04Insert(){setTitle("JDBC Insert");setSize(450,250);setDefaultCloseOperation(EXIT_ON_CLOSE);JTextField n=new
JTextField(),cse=new JTextField(),email=new JTextField();JButton save=new JButton("Save");setLayout(new
GridLayout(4,2));add(new JLabel("Name"));add(n);add(new JLabel("Course"));add(cse);add(new
JLabel("Email"));add(email);add(save);save.addActionListener(e->{String sql="INSERT INTO students(name,course,email)
VALUES(?,?,?)";try(Connection
c=DriverManager.getConnection("jdbc:mysql://localhost:3306/swing_demo","root","change_me");PreparedStatement
p=c.prepareStatement(sql)){p.setString(1,n.getText());p.setString(2,cse.getText());p.setString(3,email.getText());p.
executeUpdate();JOptionPane.showMessageDialog(this,"Inserted");}catch(SQLException
x){JOptionPane.showMessageDialog(this,x.getMessage());}});setLocationRelativeTo(null);setVisible(true);}public
static void main(String[] a){SwingUtilities.invokeLater(W13E04Insert::new);}}
```

Example 5: JDBC Table Viewer

Goal: Load MySQL rows into JTable.

```
import javax.swing.*;import javax.swing.table.DefaultTableModel;import java.awt.*;import java.sql.*;public class
W13E05TableViewer extends JFrame{DefaultTableModel model=new DefaultTableModel(new
String[]{"ID","Name","Course","Email"},0);public W13E05TableViewer(){setTitle("Database
Students");setSize(650,380);setDefaultCloseOperation(EXIT_ON_CLOSE);JButton load=new
JButton("Refresh");load.addActionListener(e->load());add(new JScrollPane(new
JTable(model)));add(load,BorderLayout.SOUTH);setLocationRelativeTo(null);setVisible(true);load();}void
load(){model.setRowCount(0);try(Connection
c=DriverManager.getConnection("jdbc:mysql://localhost:3306/swing_demo","root","change_me");Statement
s=c.createStatement();ResultSet r=s.executeQuery("SELECT * FROM students ORDER BY
id")){while(r.next())model.addRow(new
Object[]{r.getInt("id"),r.getString("name"),r.getString("course"),r.getString("email")});}catch(SQLException
x){JOptionPane.showMessageDialog(this,x.getMessage());}}public static void main(String[]
a){SwingUtilities.invokeLater(W13E05TableViewer::new);}}
```

Stage Review Questions

1. What problem does JDBC solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 14: Database CRUD

Move CRUD operations from the in-memory TableModel to persistent MySQL records.

Discussion

Key concepts: INSERT/SELECT/UPDATE/DELETE; generated data refresh; table/form synchronization; SQL exceptions; PreparedStatement.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Database Create

Goal: Insert validated form data using PreparedStatement.

```
import javax.swing.*;import java.awt.*;import java.sql.*;public class W14E01Create extends JFrame{JTextField n=new
JTextField(),c=new JTextField(),e=new JTextField();public W14E01Create(){setTitle("DB
Create");setSize(420,250);setDefaultCloseOperation(EXIT_ON_CLOSE);setLayout(new GridLayout(4,2));add(new
JLabel("Name"));add(n);add(new JLabel("Course"));add(c);add(new JLabel("Email"));add(e);JButton b=new
JButton("Insert");add(b);b.addActionListener(x-
>{if(n.getText().trim().isEmpty()){JOptionPane.showMessageDialog(this,"Name required");return;}try(Connection
cn=DriverManager.getConnection("jdbc:mysql://localhost:3306/swing_demo","root","change_me");PreparedStatement
p=cn.prepareStatement("INSERT INTO students(name,course,email)
VALUES(?,?,?)")){p.setString(1,n.getText());p.setString(2,c.getText());p.setString(3,e.getText());p.executeUpdate();
JOptionPane.showMessageDialog(this,"Saved");}catch(SQLException
z){JOptionPane.showMessageDialog(this,z.getMessage());}});setLocationRelativeTo(null);setVisible(true);}public
static void main(String[] a){SwingUtilities.invokeLater(W14E01Create::new);}}
```

Example 2: Database Read

Goal: Find one student by ID.

```
import javax.swing.*;import java.awt.*;import java.sql.*;public class W14E02Read extends JFrame{public
W14E02Read(){setTitle("DB Read");setSize(450,200);setDefaultCloseOperation(EXIT_ON_CLOSE);JTextField id=new
JTextField(6);JButton find=new JButton("Find");JLabel out=new JLabel();find.addActionListener(e->{try(Connection
c=DriverManager.getConnection("jdbc:mysql://localhost:3306/swing_demo","root","change_me");PreparedStatement
p=c.prepareStatement("SELECT * FROM students WHERE id=?")){p.setInt(1,Integer.parseInt(id.getText()));try(ResultSet
r=p.executeQuery()){out.setText(r.next()?r.getString("name")+" | "+r.getString("course"):"Not
found");}}catch(Exception x){out.setText(x.getMessage());}});JPanel p=new JPanel();p.add(new
JLabel("ID"));p.add(id);p.add(find);p.add(out);add(p);setLocationRelativeTo(null);setVisible(true);}public static
void main(String[] a){SwingUtilities.invokeLater(W14E02Read::new);}}
```

Example 3: Database Update

Goal: Update a row by ID.

```
import javax.swing.*;import java.awt.*;import java.sql.*;public class W14E03Update extends JFrame{public
W14E03Update(){setTitle("DB Update");setSize(430,220);setDefaultCloseOperation(EXIT_ON_CLOSE);JTextField id=new
JTextField(),name=new JTextField();JButton b=new JButton("Update Name");setLayout(new GridLayout(3,2));add(new
JLabel("ID"));add(id);add(new JLabel("New Name"));add(name);add(b);b.addActionListener(e->{try(Connection
c=DriverManager.getConnection("jdbc:mysql://localhost:3306/swing_demo","root","change_me");PreparedStatement
p=c.prepareStatement("UPDATE students SET name=? WHERE
id=?")){p.setString(1,name.getText());p.setInt(2,Integer.parseInt(id.getText()));JOptionPane.showMessageDialog(this,
"Rows updated: "+p.executeUpdate());}catch(Exception
```

```
x){JOptionPane.showMessageDialog(this,x.getMessage());}});setLocationRelativeTo(null);setVisible(true);}public
static void main(String[] a){SwingUtilities.invokeLater(W14E03Update::new);}}
```

Example 4: Database Delete

Goal: Delete a row with confirmation.

```
import javax.swing.*;import java.awt.*;import java.sql.*;public class W14E04Delete extends JFrame{public
W14E04Delete(){setTitle("DB Delete");setSize(400,180);setDefaultCloseOperation(EXIT_ON_CLOSE);JTextField id=new
JTextField(8);JButton b=new JButton("Delete");b.addActionListener(e->{if(JOptionPane.showConfirmDialog(this,"Delete
ID "+id.getText()+"?")==JOptionPane.YES_OPTION)try(Connection
c=DriverManager.getConnection("jdbc:mysql://localhost:3306/swing_demo","root","change_me");PreparedStatement
p=c.prepareStatement("DELETE FROM students WHERE
id=?")){p.setInt(1,Integer.parseInt(id.getText()));JOptionPane.showMessageDialog(this,"Rows deleted:
"+p.executeUpdate());}catch(Exception x){JOptionPane.showMessageDialog(this,x.getMessage());}});JPanel p=new
JPanel();p.add(new JLabel("ID"));p.add(id);p.add(b);add(p);setLocationRelativeTo(null);setVisible(true);}public
static void main(String[] a){SwingUtilities.invokeLater(W14E04Delete::new);}}
```

Example 5: Complete Database CRUD

Goal: Use JTable plus form fields for persistent CRUD.

```
import javax.swing.*;import javax.swing.table.DefaultTableModel;import java.awt.*;import java.sql.*;public class
W14E05FullCrud extends JFrame{final String
URL="jdbc:mysql://localhost:3306/swing_demo",USER="root",PASS="change_me";DefaultTableModel m=new
DefaultTableModel(new String[]{"ID","Name","Course","Email"},0);JTable t=new JTable(m);JTextField id=new
JTextField(5),name=new JTextField(10),course=new JTextField(8),email=new JTextField(12);public
W14E05FullCrud(){setTitle("Database CRUD");setSize(850,450);setDefaultCloseOperation(EXIT_ON_CLOSE);JPanel f=new
JPanel();for(java.awt.Component c:new java.awt.Component[]{new JLabel("ID"),id,new JLabel("Name"),name,new
JLabel("Course"),course,new JLabel("Email"),email})f.add(c);JButton add=new JButton("Add"),upd=new
JButton("Update"),del=new JButton("Delete"),ref=new
JButton("Refresh");f.add(add);f.add(upd);f.add(del);f.add(ref);add(new
JScrollPane(t));add(f,BorderLayout.NORTH);t.getSelectionModel().addListSelectionListener(e->{int
r=t.getSelectedRow();if(r>=0){id.setText(m.getValueAt(r,0).toString());name.setText(m.getValueAt(r,1).toString());co
urse.setText(m.getValueAt(r,2).toString());email.setText(String.valueOf(m.getValueAt(r,3)));}});add.addActionListene
r(e->sql("INSERT INTO students(name,course,email) VALUES(?,?,?)",false));upd.addActionListener(e->sql("UPDATE
students SET name=?,course=?,email=? WHERE id=?",true));del.addActionListener(e->{try(Connection
c=DriverManager.getConnection(URL,USER,PASS);PreparedStatement p=c.prepareStatement("DELETE FROM students WHERE
id=?")){p.setInt(1,Integer.parseInt(id.getText()));p.executeUpdate();load();}catch(Exception
x){msg(x);}});ref.addActionListener(e->load());setLocationRelativeTo(null);setVisible(true);load();}void sql(String
q,boolean withId){try(Connection c=DriverManager.getConnection(URL,USER,PASS);PreparedStatement
p=c.prepareStatement(q)){p.setString(1,name.getText());p.setString(2,course.getText());p.setString(3,email.getText()
);if(withId)p.setInt(4,Integer.parseInt(id.getText()));p.executeUpdate();load();}catch(Exception x){msg(x);}}void
load(){m.setRowCount(0);try(Connection c=DriverManager.getConnection(URL,USER,PASS);Statement
s=c.createStatement();ResultSet r=s.executeQuery("SELECT * FROM students ORDER BY id")){while(r.next())m.addRow(new
Object[]{r.getInt(1),r.getString(2),r.getString(3),r.getString(4)});}catch(Exception x){msg(x);}}void msg(Exception
x){JOptionPane.showMessageDialog(this,x.getMessage());}public static void main(String[]
a){SwingUtilities.invokeLater(W14E05FullCrud::new);}}
```

Stage Review Questions

1. What problem does Database CRUD solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 15: Search / Filtering

Filtering improves usability when datasets grow. Teach both client-side JTable filtering and database-side SQL search.

Discussion

Key concepts: TableRowSorter; RowFilter; DocumentListener; regex escaping; SQL LIKE; multi-criteria queries.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: JTable RowSorter

Goal: Enable automatic table sorting.

```
import javax.swing.*;import javax.swing.table.*;public class W15E01Sorter extends JFrame{public
W15E01Sorter(){Object[][] d={{1,"Ana","BSIT"},{2,"Ben","BSCS"},{3,"Cara","BSIT"}};DefaultTableModel m=new
DefaultTableModel(d,new String[]{"ID","Name","Course"});JTable t=new
JTable(m);t.setAutoCreateRowSorter(true);setTitle("Sortable
Table");setSize(500,300);setDefaultCloseOperation(EXIT_ON_CLOSE);add(new
JScrollPane(t));setLocationRelativeTo(null);setVisible(true);}public static void main(String[]
a){SwingUtilities.invokeLater(W15E01Sorter::new);}}
```

Example 2: Live Text Filter

Goal: Filter JTable rows as text changes.

```
import javax.swing.*;import javax.swing.event.*;import javax.swing.table.*;import java.awt.*;public class
W15E02LiveFilter extends JFrame{public W15E02LiveFilter(){DefaultTableModel m=new DefaultTableModel(new
Object[][]{{1,"Ana","BSIT"},{2,"Ben","BSCS"},{3,"Carla","BSEMC"}},new String[]{"ID","Name","Course"});JTable t=new
JTable(m);TableRowSorter<DefaultTableModel> sorter=new TableRowSorter<>(m);t.setRowSorter(sorter);JTextField q=new
JTextField(20);q.getDocument().addDocumentListener(new DocumentListener(){public void insertUpdate(DocumentEvent
e){f();}public void removeUpdate(DocumentEvent e){f();}public void changedUpdate(DocumentEvent e){f();}void
f(){sorter.setRowFilter(RowFilter.regexFilter("(?i)"+java.util.regex.Pattern.quote(q.getText())));}});add(q,BorderLa
yout.NORTH);add(new JScrollPane(t));setTitle("Live
Filter");setSize(500,320);setDefaultCloseOperation(EXIT_ON_CLOSE);setLocationRelativeTo(null);setVisible(true);}publ
ic static void main(String[] a){SwingUtilities.invokeLater(W15E02LiveFilter::new);}}
```

Example 3: Filter by ComboBox

Goal: Filter rows by course selection.

```
import javax.swing.*;import javax.swing.table.*;import java.awt.*;public class W15E03CourseFilter extends
JFrame{public W15E03CourseFilter(){DefaultTableModel m=new DefaultTableModel(new
Object[][]{{1,"Ana","BSIT"},{2,"Ben","BSCS"},{3,"Cara","BSIT"}},new String[]{"ID","Name","Course"});JTable t=new
JTable(m);TableRowSorter<DefaultTableModel> s=new TableRowSorter<>(m);t.setRowSorter(s);JComboBox<String> c=new
JComboBox<>(new String[]{"All","BSIT","BSCS"});c.addActionListener(e->{String
v=(String)c.getSelectedItem();s.setRowFilter(v.equals("All")?null:RowFilter.regexFilter("^"+v+"$",2));});add(c,Borde
rLayout.NORTH);add(new JScrollPane(t));setTitle("Course
Filter");setSize(500,300);setDefaultCloseOperation(EXIT_ON_CLOSE);setLocationRelativeTo(null);setVisible(true);}publ
ic static void main(String[] a){SwingUtilities.invokeLater(W15E03CourseFilter::new);}}
```

Example 4: SQL Search

Goal: Search MySQL records using LIKE.

```
import javax.swing.*;import javax.swing.table.DefaultTableModel;import java.awt.*;import java.sql.*;public class
W15E04SQLSearch extends JFrame{DefaultTableModel m=new DefaultTableModel(new
String[]{"ID","Name","Course","Email"},0);public W15E04SQLSearch(){JTable t=new JTable(m);JTextField q=new
JTextField(20);JButton b=new JButton("Search");b.addActionListener(e->{m.setRowCount(0);try(Connection
c=DriverManager.getConnection("jdbc:mysql://localhost:3306/swing_demo","root","change_me");PreparedStatement
p=c.prepareStatement("SELECT * FROM students WHERE name LIKE ? OR course LIKE ?")){String
s="%"+q.getText()+"%";p.setString(1,s);p.setString(2,s);try(ResultSet
r=p.executeQuery()){while(r.next())m.addRow(new
Object[]{r.getInt(1),r.getString(2),r.getString(3),r.getString(4)});}}catch(Exception
x){JOptionPane.showMessageDialog(this,x.getMessage());}});JPanel top=new
JPanel();top.add(q);top.add(b);add(top,BorderLayout.NORTH);add(new JScrollPane(t));setTitle("SQL
Search");setSize(650,350);setDefaultCloseOperation(EXIT_ON_CLOSE);setLocationRelativeTo(null);setVisible(true);}publ
ic static void main(String[] a){SwingUtilities.invokeLater(W15E04SQLSearch::new);}}
```

Example 5: Multi-Criteria Search

Goal: Use optional name and course criteria.

```
import javax.swing.*;import javax.swing.table.DefaultTableModel;import java.awt.*;import java.sql.*;public class
W15E05MultiSearch extends JFrame{DefaultTableModel m=new DefaultTableModel(new
String[]{"ID","Name","Course"},0);public W15E05MultiSearch(){JTable t=new JTable(m);JTextField name=new
JTextField(12);JComboBox<String> course=new JComboBox<>(new String[]{"All","BSIT","BSCS","BSEMC"});JButton
search=new JButton("Search");search.addActionListener(e->{m.setRowCount(0);String sql="SELECT id,name,course FROM
students WHERE name LIKE ? AND (?='All' OR course=?)";try(Connection
c=DriverManager.getConnection("jdbc:mysql://localhost:3306/swing_demo","root","change_me");PreparedStatement
p=c.prepareStatement(sql)){p.setString(1,"%"+name.getText()+"%");p.setString(2,(String)course.getSelectedItem());p.s
etString(3,(String)course.getSelectedItem());try(ResultSet r=p.executeQuery()){while(r.next())m.addRow(new
Object[]{r.getInt(1),r.getString(2),r.getString(3)});}}catch(Exception
x){JOptionPane.showMessageDialog(this,x.getMessage());}});JPanel top=new JPanel();top.add(new
JLabel("Name"));top.add(name);top.add(course);top.add(search);add(top,BorderLayout.NORTH);add(new
JScrollPane(t));setTitle("Multi
Search");setSize(600,350);setDefaultCloseOperation(EXIT_ON_CLOSE);setLocationRelativeTo(null);setVisible(true);}publ
ic static void main(String[] a){SwingUtilities.invokeLater(W15E05MultiSearch::new);}}
```

Stage Review Questions

1. What problem does Search / Filtering solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 16: MVC / DAO

MVC separates user interface, application behavior and data. DAO isolates persistence logic. Together they make larger Swing systems easier to test and maintain.

Discussion

Key concepts: Model; View; Controller; DAO interface; JDBC DAO; dependency separation; avoiding SQL in event handlers.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Model Class

Goal: Separate domain data into a model.

```
public class W16E01StudentModel {private int id;private String name;private String course;public
W16E01StudentModel(int id,String name,String course){this.id=id;this.name=name;this.course=course;}public int
getId(){return id;}public String getName(){return name;}public String getCourse(){return course;}public void
setName(String name){this.name=name;}public void setCourse(String course){this.course=course;}public String
toString(){return id+" "+name+" "+course;}public static void main(String[] a){System.out.println(new
W16E01StudentModel(1,"Ana","BSIT"));}}
```

Example 2: DAO Interface

Goal: Define persistence operations using an interface.

```
import java.util.*;interface StudentDAO{void add(Student s)throws Exception;List<Student> findAll()throws
Exception;}class Student{int id;String name;Student(int id,String name){this.id=id;this.name=name;}public String
toString(){return id+"-"+name;}}class MemoryStudentDAO implements StudentDAO{private final List<Student> data=new
ArrayList<>();public void add(Student s){data.add(s);}public List<Student> findAll(){return new
ArrayList<>(data);}}public class W16E02DAOInterface{public static void main(String[] a)throws Exception{StudentDAO
dao=new MemoryStudentDAO();dao.add(new Student(1,"Ana"));System.out.println(dao.findAll());}}
```

Example 3: JDBC DAO

Goal: Move SQL code out of the GUI.

```
import java.sql.*;import java.util.*;class StudentRecord{int id;String name,course;StudentRecord(int i,String
n,String c){id=i;name=n;course=c;}}class JdbcStudentDAO{final String
URL="jdbc:mysql://localhost:3306/swing_demo";List<StudentRecord> findAll()throws SQLException{List<StudentRecord>
list=new ArrayList<>();try(Connection c=DriverManager.getConnection(URL,"root","change_me");Statement
s=c.createStatement();ResultSet r=s.executeQuery("SELECT id,name,course FROM students")){while(r.next())list.add(new
StudentRecord(r.getInt(1),r.getString(2),r.getString(3)));}return list;}}public class W16E03JdbcDAO{public static
void main(String[] a)throws Exception{System.out.println(new JdbcStudentDAO().findAll().size());}}
```

Example 4: MVC In Memory

Goal: Connect model, view and controller without a DB.

```
import javax.swing.*;import java.awt.*;class CounterModel{private int value;int get(){return value;}void
inc(){value++;}}class CounterView extends JFrame{JLabel label=new JLabel("0",SwingConstants.CENTER);JButton
button=new JButton("Increment");CounterView(){setTitle("MVC
Counter");setSize(300,180);setDefaultCloseOperation(EXIT_ON_CLOSE);add(label);add(button,BorderLayout.SOUTH);setLoca
```

```
tionRelativeTo(null);}}class CounterController{CounterController(CounterModel m,CounterView
v){v.button.addActionListener(e->{m.inc();v.label.setText(String.valueOf(m.get()));});v.setVisible(true);}}public
class W16E04MVC{public static void main(String[] a){SwingUtilities.invokeLater(()->new CounterController(new
CounterModel(),new CounterView()));}}
```

Example 5: MVC + DAO Student List

Goal: Use a controller to load DAO data into a Swing view.

```
import javax.swing.*;import javax.swing.table.DefaultTableModel;import java.awt.*;import java.sql.*;class
StudentListDAO{ResultSet query(Connection c)throws SQLException{return c.createStatement().executeQuery("SELECT
id,name,course FROM students");}}class StudentListView extends JFrame{DefaultTableModel m=new DefaultTableModel(new
String[]{"ID","Name","Course"},0);JButton load=new JButton("Load");StudentListView(){setTitle("MVC DAO
Students");setSize(550,350);setDefaultCloseOperation(EXIT_ON_CLOSE);add(new JScrollPane(new
JTable(m)));add(load,BorderLayout.SOUTH);setLocationRelativeTo(null);}}class
StudentListController{StudentListController(StudentListView v,StudentListDAO d){v.load.addActionListener(e-
>{v.m.setRowCount(0);try(Connection
c=DriverManager.getConnection("jdbc:mysql://localhost:3306/swing_demo","root","change_me");ResultSet
r=d.query(c)){while(r.next())v.m.addRow(new Object[]{r.getInt(1),r.getString(2),r.getString(3)});}catch(Exception
x){JOptionPane.showMessageDialog(v,x.getMessage());}});v.setVisible(true);}}public class W16E05MVCDao{public static
void main(String[] a){SwingUtilities.invokeLater(()->new StudentListController(new StudentListView(),new
StudentListDAO()));}}
```

Stage Review Questions

1. What problem does MVC / DAO solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 17: SwingWorker

Swing is single-threaded. Long operations on the Event Dispatch Thread freeze the interface. SwingWorker moves slow work to a background thread and safely publishes results back to the GUI.

Discussion

Key concepts: EDT; doInBackground; publish/process; done/get; cancellation; progress; background file and database operations.

Instructions

1. Explain the component or design principle and where it is used in real desktop systems.
2. Demonstrate the smallest working example first, then identify the important lines.
3. Ask students to modify labels, layout, input fields, validation rules or data values.
4. Run the program and deliberately test normal input, blank input and invalid input when applicable.
5. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Background Counter

Goal: Run a slow loop without freezing the GUI.

```
import javax.swing.*;import java.awt.*;public class W17E01Worker extends JFrame{JProgressBar bar=new
JProgressBar(0,100);JButton start=new JButton("Start");public W17E01Worker(){setTitle("SwingWorker
Counter");setSize(400,180);setDefaultCloseOperation(EXIT_ON_CLOSE);add(bar);add(start,BorderLayout.SOUTH);start.addA
ctionListener(e->{start.setEnabled(false);new SwingWorker<Void,Integer>(){protected Void doInBackground()throws
Exception{for(int i=0;i<=100;i++){Thread.sleep(30);publish(i);}return null;}protected void
process(java.util.List<Integer> v){bar.setValue(v.get(v.size()-1));}protected void
done(){start.setEnabled(true);}}.execute();});setLocationRelativeTo(null);setVisible(true);}public static void
main(String[] a){SwingUtilities.invokeLater(W17E01Worker::new);}}
```

Example 2: Cancelable Worker

Goal: Cancel a background task.

```
import javax.swing.*;import java.awt.*;public class W17E02Cancel extends JFrame{JProgressBar bar=new
JProgressBar(0,100);SwingWorker<Void,Integer> worker;public W17E02Cancel(){JButton start=new
JButton("Start"),cancel=new JButton("Cancel");start.addActionListener(e->{worker=new SwingWorker<>(){protected Void
doInBackground()throws Exception{for(int i=0;i<=100&&!isCancelled();i++){Thread.sleep(50);publish(i);}return
null;}protected void process(java.util.List<Integer> x){bar.setValue(x.get(x.size()-
1));}};worker.execute();});cancel.addActionListener(e->{if(worker!=null)worker.cancel(true);});JPanel p=new
JPanel();p.add(start);p.add(cancel);setTitle("Cancelable
Worker");setSize(400,180);setDefaultCloseOperation(EXIT_ON_CLOSE);add(bar);add(p,BorderLayout.SOUTH);setLocationRela
tiveTo(null);setVisible(true);}public static void main(String[] a){SwingUtilities.invokeLater(W17E02Cancel::new);}}
```

Example 3: Worker Result

Goal: Return a computed result from doInBackground.

```
import javax.swing.*;import java.awt.*;public class W17E03Result extends JFrame{JLabel out=new
JLabel("Ready",SwingConstants.CENTER);public W17E03Result(){JButton b=new JButton("Compute");b.addActionListener(e-
>{new SwingWorker<Long,Void>(){protected Long doInBackground(){long sum=0;for(int
i=1;i<=10_000_000;i++)sum+=i;return sum;}protected void done(){try{out.setText("Result: "+get());}catch(Exception
x){out.setText(x.getMessage());}}}.execute();});setTitle("Worker
Result");setSize(450,180);setDefaultCloseOperation(EXIT_ON_CLOSE);add(out);add(b,BorderLayout.SOUTH);setLocationRela
tiveTo(null);setVisible(true);}public static void main(String[] a){SwingUtilities.invokeLater(W17E03Result::new);}}
```

Example 4: Background File Load

Goal: Load a large text file in the background.

```
import javax.swing.*;import java.awt.*;import java.nio.file.*;public class W17E04FileLoad extends JFrame{JTextArea
area=new JTextArea();public W17E04FileLoad(){JButton b=new JButton("Load big.txt");b.addActionListener(e-
>{b.setEnabled(false);new SwingWorker<String,Void>(){protected String doInBackground()throws Exception{return
Files.readString(Path.of("big.txt"));}protected void done(){try{area.setText(get());}catch(Exception
x){area.setText(x.getMessage());}b.setEnabled(true);}}.execute();});setTitle("Background File
Load");setSize(550,380);setDefaultCloseOperation(EXIT_ON_CLOSE);add(new
JScrollPane(area));add(b,BorderLayout.SOUTH);setLocationRelativeTo(null);setVisible(true);}public static void
main(String[] a){SwingUtilities.invokeLater(W17E04FileLoad::new);}}
```

Example 5: Background Database Load

Goal: Load database rows without blocking the EDT.

```
import javax.swing.*;import javax.swing.table.DefaultTableModel;import java.awt.*;import java.sql.*;import
java.util.*;public class W17E05DbWorker extends JFrame{DefaultTableModel m=new DefaultTableModel(new
String[]{"ID","Name","Course"},0);public W17E05DbWorker(){JTable t=new JTable(m);JButton b=new JButton("Load
Database");b.addActionListener(e->{b.setEnabled(false);new SwingWorker<java.util.List<Object[]>,Void>(){protected
java.util.List<Object[]> doInBackground()throws Exception{java.util.List<Object[]> list=new
ArrayList<>();try(Connection
c=DriverManager.getConnection("jdbc:mysql://localhost:3306/swing_demo","root","change_me");Statement
s=c.createStatement();ResultSet r=s.executeQuery("SELECT id,name,course FROM students")){while(r.next())list.add(new
Object[]{r.getInt(1),r.getString(2),r.getString(3)});}return list;}protected void
done(){m.setRowCount(0);try{for(Object[] row:get())m.addRow(row);}catch(Exception
x){JOptionPane.showMessageDialog(W17E05DbWorker.this,x.getMessage());}b.setEnabled(true);}}.execute();});setTitle("D
B SwingWorker");setSize(600,350);setDefaultCloseOperation(EXIT_ON_CLOSE);add(new
JScrollPane(t));add(b,BorderLayout.SOUTH);setLocationRelativeTo(null);setVisible(true);}public static void
main(String[] a){SwingUtilities.invokeLater(W17E05DbWorker::new);}}
```

Stage Review Questions

1. What problem does SwingWorker solve in a Swing application?
2. Which events or methods are most important in these examples?
3. What invalid user actions should the application prevent?
4. How would you improve the layout and usability of one example?
5. How can this stage be combined with the previous stage in a larger system?

Stage 18: Complete Information System

The capstone stage combines interface design, validation, domain models, collections, persistence, database access, search, architecture and responsive background work.

Discussion

Key concepts: requirements; modular forms; validation; persistence; CRUD; search; MVC/DAO; SwingWorker; maintainability; error handling.

Instructions

6. Explain the component or design principle and where it is used in real desktop systems.
7. Demonstrate the smallest working example first, then identify the important lines.
8. Ask students to modify labels, layout, input fields, validation rules or data values.
9. Run the program and deliberately test normal input, blank input and invalid input when applicable.
10. End the stage by asking students to combine at least two examples into one small application.

Examples

Example 1: Student Information System - In Memory

Goal: Integrate validation, models, ArrayList, JTable and search.

```
import javax.swing.*;import javax.swing.table.*;import java.awt.*;import java.util.*;class SISStudent{int id;String
name,course;SISStudent(int i,String n,String c){id=i;name=n;course=c;}}public class W18E01StudentSystem extends
JFrame{ArrayList<SISStudent> data=new ArrayList<>();DefaultTableModel m=new DefaultTableModel(new
String[]{"ID","Name","Course"},0);JTable t=new JTable(m);JTextField id=new JTextField(5),name=new
JTextField(10),course=new JTextField(8),search=new JTextField(10);public W18E01StudentSystem(){setTitle("Student
Information System");setSize(800,450);setDefaultCloseOperation(EXIT_ON_CLOSE);JPanel top=new JPanel();for(Component
c:new Component[]{new JLabel("ID"),id,new JLabel("Name"),name,new JLabel("Course"),course})top.add(c);JButton
add=new JButton("Add"),del=new JButton("Delete");top.add(add);top.add(del);JPanel south=new JPanel();JButton
find=new JButton("Search");south.add(new JLabel("Name
contains"));south.add(search);south.add(find);add.addActionListener(e-
>{if(name.getText().trim().isEmpty()){JOptionPane.showMessageDialog(this,"Name required");return;}SISStudent s=new
SISStudent(Integer.parseInt(id.getText()),name.getText(),course.getText());data.add(s);refresh(data);});del.addActio
nListener(e->{int r=t.getSelectedRow();if(r>=0){data.remove(r);refresh(data);}});find.addActionListener(e->{String
q=search.getText().toLowerCase();ArrayList<SISStudent> out=new ArrayList<>();for(SISStudent
s:data)if(s.name.toLowerCase().contains(q))out.add(s);refresh(out);});add(top,BorderLayout.NORTH);add(new
JScrollPane(t));add(south,BorderLayout.SOUTH);setLocationRelativeTo(null);setVisible(true);}void
refresh(java.util.List<SISStudent> list){m.setRowCount(0);for(SISStudent s:list)m.addRow(new
Object[]{s.id,s.name,s.course});}public static void main(String[]
a){SwingUtilities.invokeLater(W18E01StudentSystem::new);}}
```

Example 2: Inventory Information System

Goal: Create a complete in-memory inventory with totals.

```
import javax.swing.*;import javax.swing.table.*;import java.awt.*;import java.util.*;class Item{String code,name;int
qty;double price;Item(String c,String n,int q,double p){code=c;name=n;qty=q;price=p;}double total(){return
qty*price;}}public class W18E02Inventory extends JFrame{ArrayList<Item> items=new ArrayList<>();DefaultTableModel
m=new DefaultTableModel(new String[]{"Code","Name","Qty","Price","Total"},0);JTextField code=new
JTextField(6),name=new JTextField(10),qty=new JTextField(5),price=new JTextField(6);JLabel grand=new JLabel("Grand
Total: P0.00");public W18E02Inventory(){setTitle("Inventory
System");setSize(850,450);setDefaultCloseOperation(EXIT_ON_CLOSE);JPanel f=new JPanel();for(Component c:new
Component[]{new JLabel("Code"),code,new JLabel("Name"),name,new JLabel("Qty"),qty,new
JLabel("Price"),price})f.add(c);JButton add=new JButton("Add Item");f.add(add);add.addActionListener(e-
>{try{items.add(new
Item(code.getText(),name.getText(),Integer.parseInt(qty.getText()),Double.parseDouble(price.getText())));refresh();}
catch(Exception x){JOptionPane.showMessageDialog(this,"Invalid values");}});add(f,BorderLayout.NORTH);add(new
JScrollPane(new JTable(m)));add(grand,BorderLayout.SOUTH);setLocationRelativeTo(null);setVisible(true);}void
refresh(){m.setRowCount(0);double sum=0;for(Item i:items){m.addRow(new
Object[]{i.code,i.name,i.qty,i.price,i.total()});sum+=i.total();}grand.setText(String.format("Grand Total:
P%.2f",sum));}public static void main(String[] a){SwingUtilities.invokeLater(W18E02Inventory::new);}}
```

Example 3: Contact Management System with File Persistence

Goal: Integrate forms, table and CSV persistence.

```
import javax.swing.*;import javax.swing.table.*;import java.awt.*;import java.nio.file.*;import java.util.*;public
class W18E03Contacts extends JFrame{DefaultTableModel m=new DefaultTableModel(new
String[]{"Name","Phone","Email"},0);JTextField name=new JTextField(10),phone=new JTextField(10),email=new
JTextField(12);Path file=Path.of("contacts.csv");public W18E03Contacts(){setTitle("Contact
Manager");setSize(700,400);setDefaultCloseOperation(EXIT_ON_CLOSE);JTable t=new JTable(m);JPanel f=new
JPanel();for(Component c:new Component[]{name,phone,email})f.add(c);JButton add=new JButton("Add"),save=new
JButton("Save"),load=new JButton("Load");f.add(add);f.add(save);f.add(load);add.addActionListener(e->m.addRow(new
Object[]{name.getText(),phone.getText(),email.getText()}));save.addActionListener(e->{try{java.util.List<String>
lines=new ArrayList<>();for(int
r=0;r<m.getRowCount();r++)lines.add(m.getValueAt(r,0)+","+m.getValueAt(r,1)+","+m.getValueAt(r,2));Files.write(file,
lines);JOptionPane.showMessageDialog(this,"Saved");}catch(Exception
x){JOptionPane.showMessageDialog(this,x.getMessage());}});load.addActionListener(e->{m.setRowCount(0);try{for(String
line:Files.readAllLines(file)){String[] p=line.split(",",-1);if(p.length>=3)m.addRow(p);}}catch(Exception
x){JOptionPane.showMessageDialog(this,x.getMessage());}});add(f,BorderLayout.NORTH);add(new
JScrollPane(t));setLocationRelativeTo(null);setVisible(true);}public static void main(String[]
a){SwingUtilities.invokeLater(W18E03Contacts::new);}}
```

Example 4: Database Student Information System

Goal: Integrate JDBC, JTable CRUD, validation and search.

```
import javax.swing.*;import javax.swing.table.*;import java.awt.*;import java.sql.*;public class
W18E04DbStudentSystem extends JFrame{final String
URL="jdbc:mysql://localhost:3306/swing_demo",USER="root",PASS="change_me";DefaultTableModel m=new
DefaultTableModel(new String[]{"ID","Name","Course","Email"},0);JTable t=new JTable(m);JTextField id=new
JTextField(4),name=new JTextField(10),course=new JTextField(8),email=new JTextField(12),q=new JTextField(10);public
W18E04DbStudentSystem(){setTitle("Database Student Information
System");setSize(900,480);setDefaultCloseOperation(EXIT_ON_CLOSE);JPanel f=new JPanel();for(Component c:new
Component[]{new JLabel("ID"),id,new JLabel("Name"),name,new JLabel("Course"),course,new
JLabel("Email"),email})f.add(c);JButton add=new JButton("Add"),upd=new JButton("Update"),del=new
JButton("Delete");f.add(add);f.add(upd);f.add(del);JPanel s=new JPanel();JButton search=new
JButton("Search"),all=new JButton("Show All");s.add(q);s.add(search);s.add(all);add.addActionListener(e-
>{if(name.getText().trim().isEmpty()){msg("Name required");return;}exec("INSERT INTO students(name,course,email)
VALUES(?,?,?)",false);});upd.addActionListener(e->exec("UPDATE students SET name=?,course=?,email=? WHERE
id=?",true));del.addActionListener(e->{try(Connection c=DriverManager.getConnection(URL,USER,PASS);PreparedStatement
p=c.prepareStatement("DELETE FROM students WHERE
id=?")){p.setInt(1,Integer.parseInt(id.getText()));p.executeUpdate();load("");}catch(Exception
x){msg(x.getMessage());}});search.addActionListener(e->load(q.getText()));all.addActionListener(e-
>load(""));t.getSelectionModel().addListSelectionListener(e->{int
r=t.getSelectedRow();if(r>=0){id.setText(m.getValueAt(r,0).toString());name.setText(m.getValueAt(r,1).toString());co
urse.setText(m.getValueAt(r,2).toString());email.setText(String.valueOf(m.getValueAt(r,3)));}});add(f,BorderLayout.N
ORTH);add(new JScrollPane(t));add(s,BorderLayout.SOUTH);setLocationRelativeTo(null);setVisible(true);load("");}void
exec(String sql,boolean hasId){try(Connection c=DriverManager.getConnection(URL,USER,PASS);PreparedStatement
p=c.prepareStatement(sql)){p.setString(1,name.getText());p.setString(2,course.getText());p.setString(3,email.getText
());if(hasId)p.setInt(4,Integer.parseInt(id.getText()));p.executeUpdate();load("");}catch(Exception
x){msg(x.getMessage());}}void load(String term){m.setRowCount(0);try(Connection
c=DriverManager.getConnection(URL,USER,PASS);PreparedStatement p=c.prepareStatement("SELECT * FROM students WHERE
name LIKE ? OR course LIKE ? ORDER BY id")){String s="%"+term+"%";p.setString(1,s);p.setString(2,s);try(ResultSet
r=p.executeQuery()){while(r.next())m.addRow(new
Object[]{r.getInt(1),r.getString(2),r.getString(3),r.getString(4)});}}catch(Exception x){msg(x.getMessage());}}void
msg(String x){JOptionPane.showMessageDialog(this,x);}public static void main(String[]
a){SwingUtilities.invokeLater(W18E04DbStudentSystem::new);}}
```

Example 5: Layered MVC/DAO Information System

Goal: Demonstrate a small but complete layered architecture.

```
import javax.swing.*;import javax.swing.table.*;import java.awt.*;import java.sql.*;import java.util.*;class
SysStudent{int id;String name,course;SysStudent(int i,String n,String c){id=i;name=n;course=c;}}class SysDAO{final
String URL="jdbc:mysql://localhost:3306/swing_demo";java.util.List<SysStudent> all()throws
SQLException{java.util.List<SysStudent> x=new ArrayList<>();try(Connection
c=DriverManager.getConnection(URL,"root","change_me");Statement s=c.createStatement();ResultSet
r=s.executeQuery("SELECT id,name,course FROM students ORDER BY id")){while(r.next())x.add(new
SysStudent(r.getInt(1),r.getString(2),r.getString(3)));}return x;}void add(SysStudent st)throws
SQLException{try(Connection c=DriverManager.getConnection(URL,"root","change_me");PreparedStatement
p=c.prepareStatement("INSERT INTO students(name,course)
VALUES(?,?)")){p.setString(1,st.name);p.setString(2,st.course);p.executeUpdate();}}}class SysView extends
```

```
JFrame{DefaultTableModel m=new DefaultTableModel(new String[]{"ID","Name","Course"},0);JTextField name=new
JTextField(12),course=new JTextField(8);JButton add=new JButton("Add"),refresh=new
JButton("Refresh");SysView(){setTitle("Layered Information
System");setSize(650,380);setDefaultCloseOperation(EXIT_ON_CLOSE);JPanel top=new JPanel();top.add(new
JLabel("Name"));top.add(name);top.add(new
JLabel("Course"));top.add(course);top.add(add);top.add(refresh);add(top,BorderLayout.NORTH);add(new JScrollPane(new
JTable(m)));setLocationRelativeTo(null);}}class SysController{SysView v;SysDAO d;SysController(SysView v,SysDAO
d){this.v=v;this.d=d;v.add.addActionListener(e-
>{if(v.name.getText().trim().isEmpty()){JOptionPane.showMessageDialog(v,"Name required");return;}try{d.add(new
SysStudent(0,v.name.getText(),v.course.getText()));load();}catch(Exception
x){JOptionPane.showMessageDialog(v,x.getMessage());}});v.refresh.addActionListener(e-
>load());v.setVisible(true);load();}void load(){new SwingWorker<java.util.List<SysStudent>,Void>(){protected
java.util.List<SysStudent> doInBackground()throws Exception{return d.all();}protected void
done(){try{v.m.setRowCount(0);for(SysStudent s:get())v.m.addRow(new Object[]{s.id,s.name,s.course});}catch(Exception
x){JOptionPane.showMessageDialog(v,x.getMessage());}}}.execute();}}public class W18E05LayeredSystem{public static
void main(String[] a){SwingUtilities.invokeLater(()->new SysController(new SysView(),new SysDAO()));}}
```

Stage Review Questions

11. What problem does Complete Information System solve in a Swing application?
12. Which events or methods are most important in these examples?
13. What invalid user actions should the application prevent?
14. How would you improve the layout and usability of one example?
15. How can this stage be combined with the previous stage in a larger system?

Appendix A: NetBeans Workflow

16. Create a project: File > New Project > Java with Ant (or Maven) > Java Application.
17. Create one class per example. The public class name and filename must match exactly.
18. Run a file with Shift+F6 or run the project with F6.
19. For GUI Builder practice, create JFrame Form and drag Swing controls from the Palette. Study the generated layout code, but keep application logic in your own methods/classes.
20. For MySQL Connector/J, right-click Libraries > Add JAR/Folder when using a simple Ant project, or declare mysql-connector-j in Maven/Gradle for managed builds.

Appendix B: Swing Best Practices

- Create and update Swing components on the Event Dispatch Thread.
- Use layout managers instead of absolute positioning for resizable, portable interfaces.
- Validate input before parsing or saving.
- Use PreparedStatement for values in SQL queries; never build SQL by concatenating untrusted input.
- Use try-with-resources for JDBC and file resources.
- Keep domain models, DAO/database code and GUI code separate as applications grow.
- Use SwingWorker for long-running file, network or database operations.
- Do not store production database passwords directly in source code; use configuration or environment variables.
- Give users clear feedback for success, validation errors and exceptional failures.
- Prefer one shared application architecture over duplicating connection and CRUD code in every form.

Appendix C: Suggested Capstone Requirements

- Login or role-aware landing form (optional for a classroom project).
- Dashboard with navigation menu.
- At least two domain modules (for example Students and Courses, Products and Suppliers, or Patients and Appointments).
- JTable listing with Create, Read, Update and Delete operations.
- Field validation and confirmation before destructive operations.
- Search and filter functions.
- MySQL persistence through JDBC using DAO classes.
- MVC or similar separation of concerns.
- Background loading with SwingWorker where a query or report may take time.
- File export or backup feature.
- Consistent naming, reusable methods, comments and exception handling.

Appendix D: Recommended Exercises

21. Build a student registration form with required-field, numeric and email validation.
22. Build a gradebook JTable that can add, edit and delete student scores.
23. Build a product inventory with quantity, unit price and computed total value.

24. Build a multi-form library system with Books and Borrowers menus.
25. Save and reload records using CSV before converting the same project to JDBC.
26. Create Student and Course model classes and store objects in ArrayList.
27. Convert the ArrayList-based program into a database-backed DAO implementation.
28. Add live search to the JTable.
29. Move a slow database load to SwingWorker and display progress.
30. Build a final information system using the Stage 18 architecture.